Learning Latent Dynamics For Planning from Pixels

Danijar Hafner . Timothy Lillicrap . Ian Fischer . Ruben Villegas . David Ha. Honglak Lee . James Davidson

Presented By

Emmanuel Olateju

Overview

Planning In Unknown Environments- Challenges

1. ***Learning accurate models***
2. ***Accumulating errors in multi-step predictions***
3. *Failure to capture multiple possible futures*
4. *Overconfident predictions outside training distribution*

Planning With Learned Dynamics - Advantages

1. ***More data efficient — richer training signal, no Bellman backups needed***
2. *Performance scales with compute budget for action search*
3. ***Learned dynamics are task-independent — potential for transfer***

Limitations of Existing Methods

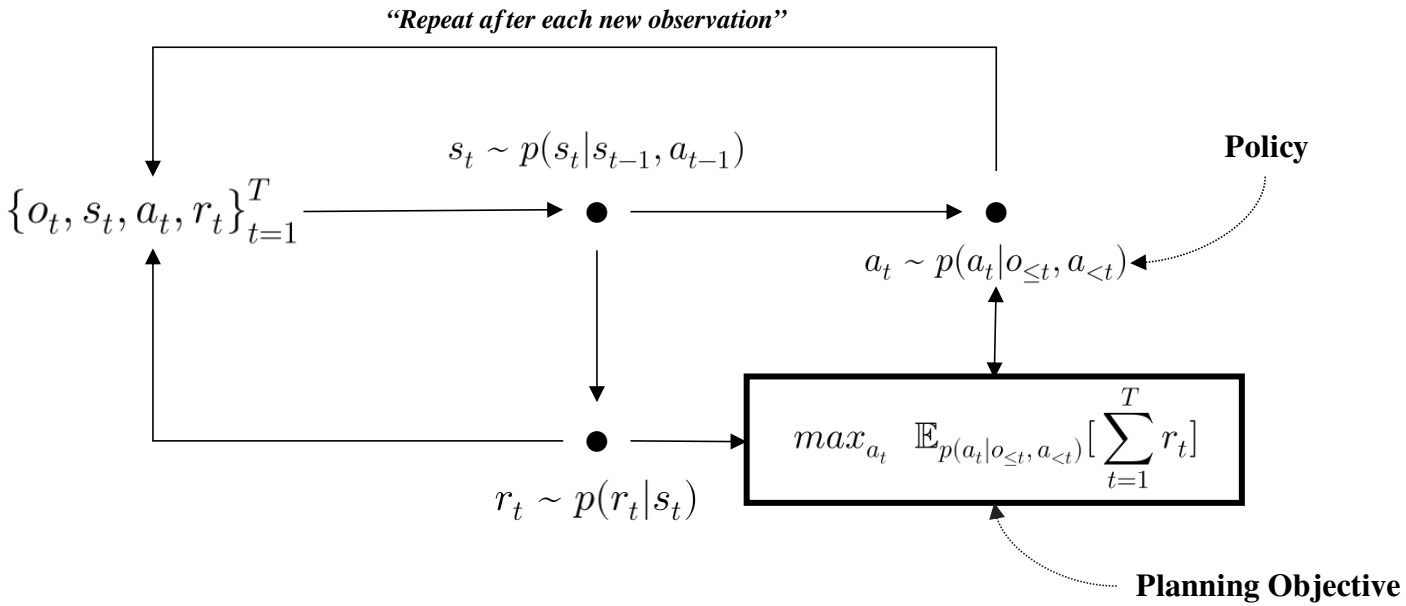
1. ***Prior work assumes access to low-dimensional ground truth state of world and reward functions — not available from pixels***
2. ***Success limited to simple tasks — cartpoles, 2-link arms, dense rewards only***
3. *Latent space models previously couldn't scale to contact dynamics, sparse rewards, or partial observability*

"Can we learn a dynamics model purely from pixel observations that is accurate enough for planning?"

PlaNet

1. *Learning of environment dynamics from images*
2. ***Fast online planning in latent space***
3. ***Multi-step prediction of rewards***
4. *Latent overshooting - variational inference*

Model-Predictive Control (MPC)



MPC Summary
1. *Fixed starting point or state*
2. *Predict future behavior using dynamic system model (transition model)*
3. *Optimize action sequences (over a planning horizon) to maximize expected cumulative reward*

Data Collection
1. *Initial S-seed episodes collected using random actions.*
2. *Model training on collected experience*
3. *New episode added to dataset every C training steps*

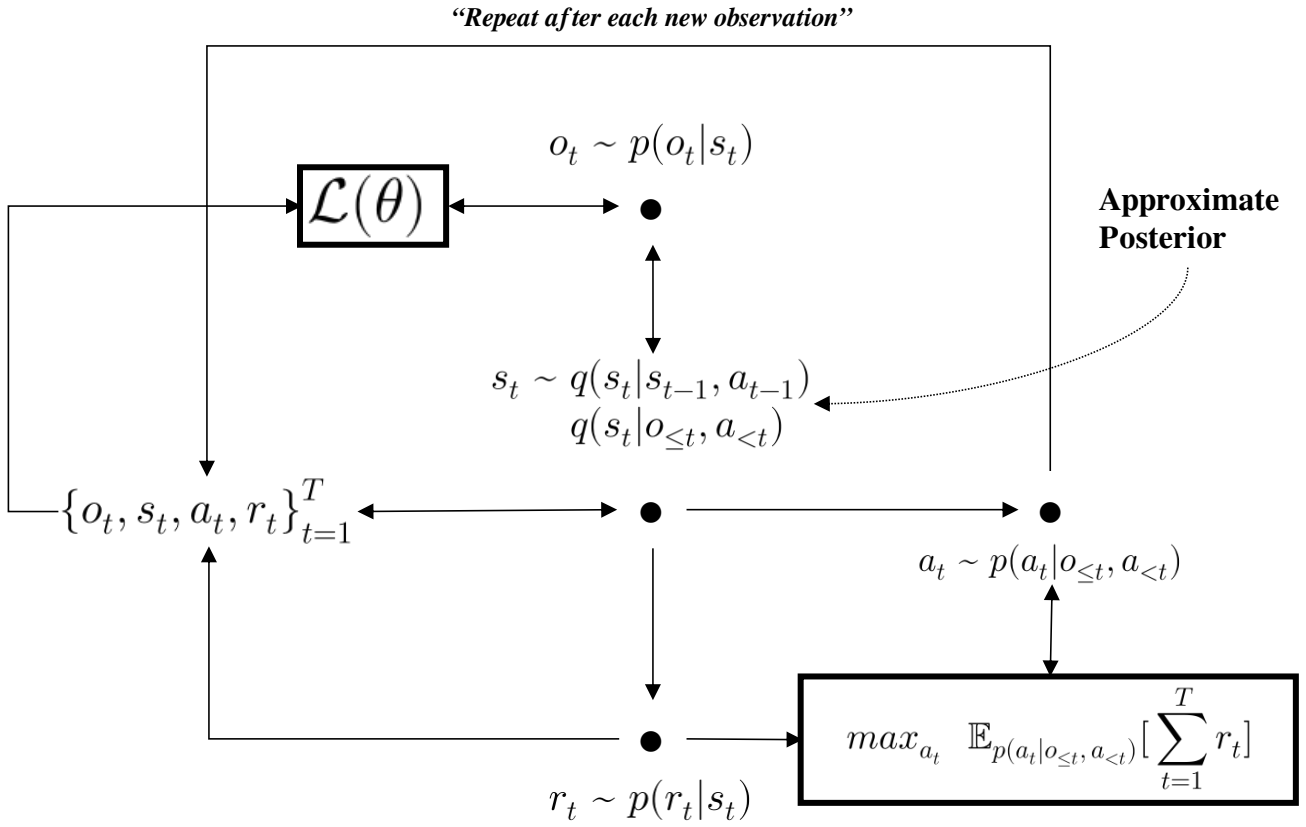
Algorithm 1: Deep Planning Network (PlaNet)

Input :

R	Action repeat	$p(s_t \mid s_{t-1}, a_{t-1})$	Transition model
S	Seed episodes	$p(o_t \mid s_t)$	Observation model
C	Collect interval	$p(r_t \mid s_t)$	Reward model
B	Batch size	$q(s_t \mid o_{\leq t}, a_{<t})$	Encoder
L	Chunk length	$p(\epsilon)$	Exploration noise
α	Learning rate		

Initialize dataset $\mathcal{D}$ with S random seed episodes.
Initialize model parameters θ randomly.
while *not converged* **do**
  `// Model fitting`
  for *update step* $s = 1..C$ **do**
    Draw sequence chunks $\{(o_t, a_t, r_t)_{t=k}^{L+k}\}_{i=1}^{B} \sim \mathcal{D}$ uniformly at random from the dataset.
    Compute loss $\mathcal{L}(\theta)$ from Equation 3.
    Update model parameters $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$.
  `// Data collection`
  $o_1 \leftarrow$ `env.reset()`
  for *time step* $t = 1..\lceil \frac{T}{R} \rceil$ **do**
    Infer belief over current state $q(s_t \mid o_{\leq t}, a_{<t})$ from the history.
    $a_t \leftarrow$ `planner`$(q(s_t \mid o_{\leq t}, a_{<t}), p)$, see Algorithm 2 in the appendix for details.
    Add exploration noise $\epsilon \sim p(\epsilon)$ to the action.
    for *action repeat* $k = 1..R$ **do**
      $r_t^k, o_{t+1}^k \leftarrow$ `env.step`(a_t)
    $r_t, o_{t+1} \leftarrow \sum_{k=1}^{R} r_t^k, o_{t+1}^R$
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o_t, a_t, r_t)_{t=1}^{T}\}$

PlaNet



PlaNet Summary
1. *Trained using a VAE-style variational objective (encoder + decoder)*
2. ***Encoder infers approximate posterior state belief from observation history***
3. ***Learned latent dynamics model predicts future states*** *without generating images* ***enabling fast planning in latent space***
4. *Observation model provides a rich training signal but is excluded from planning*

Models
1. ***Transition Model (RSSM): Deterministic (RNN) + Stochastic (Gaussian) Path***
2. *Encoder (Approximate Posterior): Gaussian, CNN + FFN*
3. *Observation Model: Gaussian, deconvolutional-Net (transposed CNN)*
4. *Reward Model: Scalar Gaussian, FFN*

One Step Prediction →

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \Big(\underbrace{\mathbb{E}_{q(s_t|o_{\leq t}, a_{<t})}[\ln p(o_t|s_t)]}_{\text{reconstruction}} - \underbrace{\mathbb{E}_{q(s_{t-1}|o_{\leq t-1}, a_{<t-1})}\left[\mathrm{KL}[q(s_t|o_{\leq t}, a_{<t}) \| p(s_t|s_{t-1}, a_{t-1})]\right]}_{\text{complexity}} \Big)$$

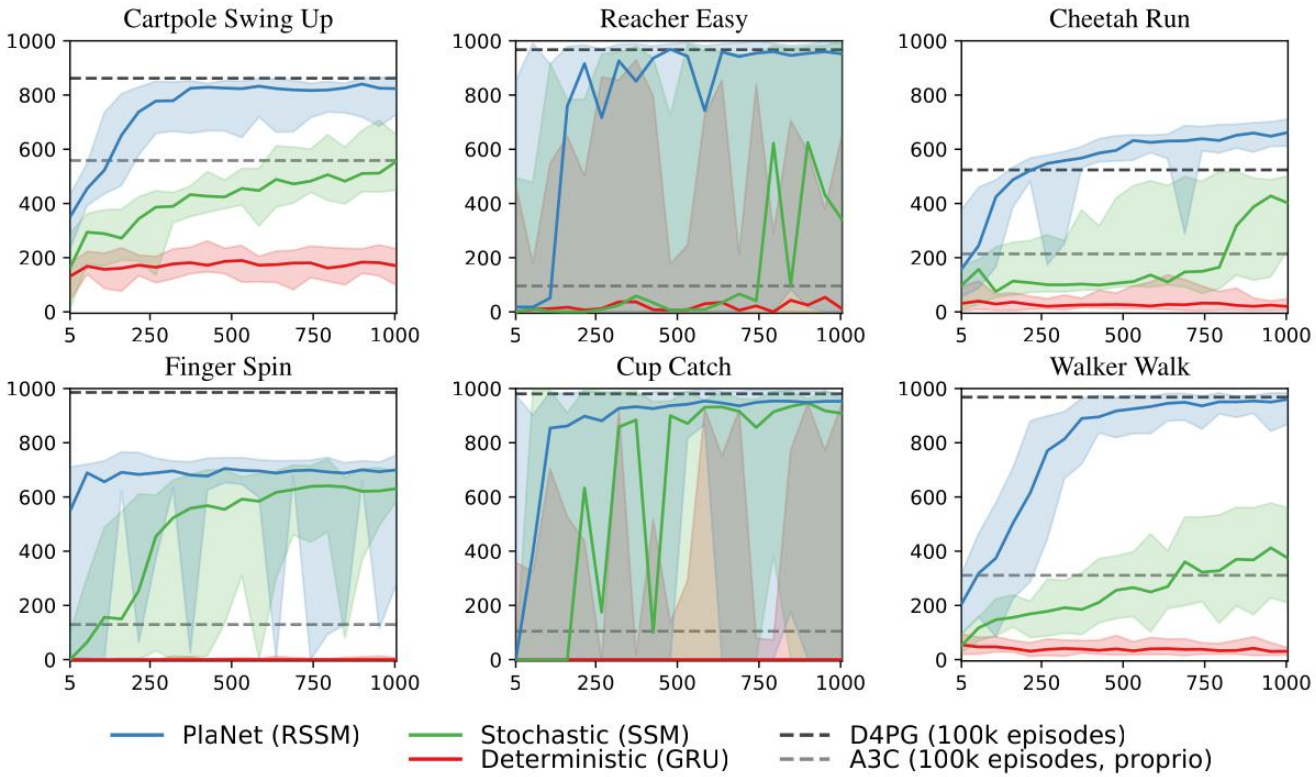
Latent Overshooting →

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \Big(\underbrace{\mathbb{E}_{q(s_t|o_{\leq t}, a_{<t})}[\ln p(o_t|s_t)]}_{\text{reconstruction}} - \underbrace{\frac{1}{D}\sum_{d=1}^{D} \beta_d \, \mathbb{E}_{q(s_{t-d}|o_{\leq t-d})p(s_{t-1}|s_{t-d})}\left[\mathrm{KL}[q(s_t|o_{\leq t}, a_{<t}) \| p(s_t|s_{t-1}, a_{t-1})]\right]}_{\text{latent overshooting}} \Big)$$

Results

Method	Modality	Episodes	Cartpole Swing Up	Reacher Easy	Cheetah Run	Finger Spin	Cup Catch	Walker Walk
A3C	proprioceptive	100,000						
D4PG	pixels	100,000						
PlaNet (ours)	pixels	1,000						
Data efficiency gain PlaNet over D4PG (factor)			250	40	500+	300	100	90

PlaNet vs Model-Free Algorithms



Key Takeaways

1. ***PlaNet matches D4PG performance using 100× fewer episodes on average***
2. ***Outperforms A3C on all tasks within just 100 episodes — A3C needed 100,000***
3. *Cheetah Run: PlaNet surpasses D4PG by 26% — the only task where model-based explicitly beats model-free*
4. ***Both stochastic and deterministic paths are critical — pure GRU and pure SSM both fail to match RSSM***
5. *Finger Spin is the weakest result — PlaNet does not match D4PG here, showing contact-heavy tasks remain challenging*

Limitations and Future Work

Limitations
1. ***Finger Spin shows contact-heavy tasks remain difficult for latent planning***
2. ***Planning horizon is fixed — no temporal abstraction due to cross-emtropy method used.***
3. ***Latent overshooting slightly reduces RSSM performance — benefit is model-dependent (helps simpler DRNN, hurts RSSM)***
4. *Limited visual diversity: cannot learn representations without reconstruction, reconstruction may model irrelevant visual features*

Future Work
1. ***Learn temporal abstraction through hierarchical models instead of fixed action repeat***
2. ***Learn a value function to approximate rewards beyond the planning horizon - DREAMER***
3. ***Gradient-based planning for better computational efficiency - cross entropy method is a population based search approach***
4. *Learn representations without reconstruction to handle higher visual diversity*
5. *Multi-task control by sharing the dynamics model across tasks*

THANK YOU